

HUMAN-IN-THE-LOOP ROBOT LEARNING

System Manual

FFW SG2 follower A2 leader

SCOPE. The architecture of the teleoperation, policy and orchestration sub-systems; the procedure for conducting an HG-Dagger session; the contents of the resulting dataset; and the training and verification procedures that follow from it.

Contents

1	System overview	1
1.1	Shared command topic	1
1.2	Session structure	1
2	Software Requirements	2
2.1	Repositories	2
2.2	Branch selection	2
2.3	Working copies that already contain a <code>feature-a2</code>	2
3	Installing the training environment	4
3.1	Memory requirement	4
3.2	Procedure	4
3.3	The base policy	4
4	Architecture	5
5	Conducting a session	8
5.1	Bring-up	8
5.2	Intervention procedure	9
5.3	Minimum correction length	12
5.4	Prohibited actions	12
6	Recorded data	13
6.1	Dimensions	14
6.2	Image preparation	14
6.3	Why neither method is applied uniformly	15
6.4	The applied configuration	16
6.5	Effect on held-out accuracy	17
6.6	Attention over the prepared views	17
7	The intervention label	20
7.1	Derivation	20
7.2	Emitted columns	20
8	HG-Dagger training	21
8.1	Procedure	21

8.2	Dataset partition	22
8.3	Warm start	22
8.4	Step budget	23
8.5	Checkpoint selection	23
9	Deployment	24
9.1	Transferring the checkpoint	24
9.2	Configuration that must match training	24
9.3	Bringing the policy up	25
9.4	Control parameters	25
9.5	Evaluation on hardware	26
10	Verification	29
10.1	Dataset validation	29
10.2	Post-conversion assertion	29
11	Topic and service reference	30
11.1	Notes	30
12	Fault diagnosis	32
A	Glossary	32
B	Reference quantities	33
C	Links	33

1 System overview

The system comprises three containers and a browser client.

Component	Responsibility
<code>ai_worker</code>	Hardware. Leader and follower arms, head, lift, swerve base, four cameras.
<code>groot_server</code>	Policy inference. GR00T-N1.7 on a TensorRT DiT backend.
<code>cyclo_intelligence</code>	Orchestration, recording, dataset conversion, external interfaces.
Browser	Operator interface for policy control, intervention and labeling.

The policy commands sixteen dimensions, corresponding to the two arms. The head, lift and mobile base are never commanded by the policy ($22 - 6 = 16$).

1.1 Shared command topic

On the SG2, the teleoperation node and the policy publish to the same arm-command topic. There is no separate policy channel, which has two consequences:

1. The recorded `action` column is valid irrespective of which agent was driving, since both write to the topic that is recorded.
2. Mutual exclusion must be enforced externally. A latched flag (T14) suppresses the leader chain while the policy is active.

1.2 Session structure

The policy executes the task autonomously. When its behavior is incorrect, the operator suspends it, corrects the motion by teleoperation, and returns control. The corrected frames constitute the training signal. The purpose of the surrounding system is to make this exchange safe and to record which frames originated from the operator.

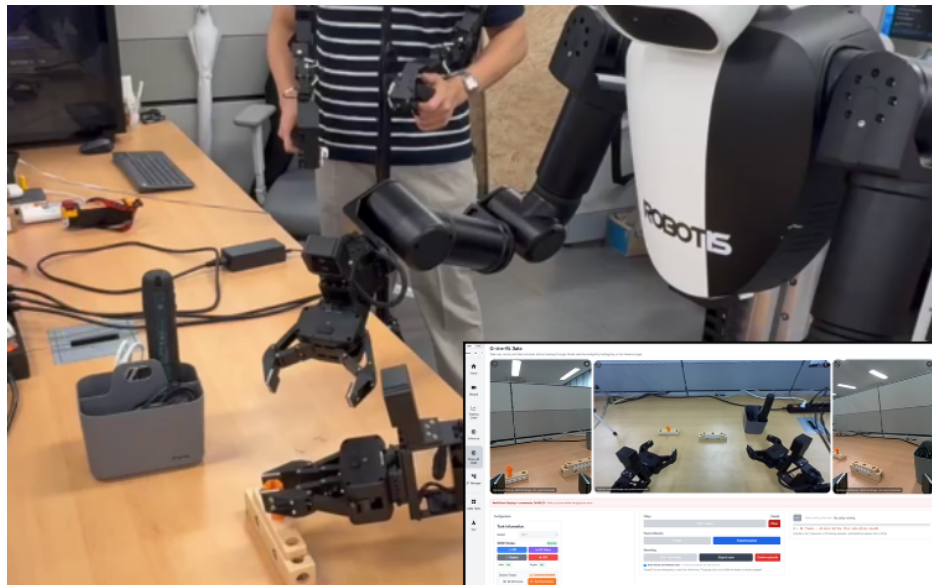


Figure 1: Work cell during a session: follower arms engaged on the task, leader arms held by the operator, operator UI.

2 Software Requirements

2.1 Repositories

Repository	Branch	Contents
da-dawit/aiw_hg_dagger	feature-a2	Robot-side packages, A2 leader support
da-dawit/cyclo_intelligence_hg_dagger	main	Orchestration, recording, conversion
da-dawit/hg-dagger	main	Training, evaluation and dataset tooling

All three are public and can be cloned over HTTPS.

```
git clone https://github.com/da-dawit/aiw_hg_dagger.git
git clone https://github.com/da-dawit/cyclo_intelligence_hg_dagger.git
git clone https://github.com/da-dawit/hg-dagger.git
```

hg-dagger contains the scripts referenced throughout Sections 8 to 10:

Path	Purpose
scripts/verify_dataset.py	Dataset validation gate
scripts/fix_dagger_dataset.py	Repair a converted online-RL dataset
scripts/split_dagger.py	Partition by intervention label
configs/export_dagger_split.py	Write the partition as LeRobot datasets
configs/06_train_dagger.sh	HG-Dagger training launcher
scripts/checkpoint_sweep.py	Held-out error per checkpoint
scripts/rollout3d.py	Cartesian rollout evaluation
scripts/attn_map.py	Attention diagnostics
scripts/crop_stats.py	Motion energy and detail per crop region
configs/roi_hybrid.json	Per-camera crop specification
configs/crop_to_square.patch	Required patch to Isaac-GR00T: per-camera crop
configs/mixture_ratio.patch	Required patch to Isaac-GR00T: dataset mixture ratios

2.2 Branch selection

aiw_hg_dagger carries two branches. The work described in this manual is on **feature-a2**, which is the default and is 32 commits ahead of **ai_worker-wip**. A plain clone therefore checks out the correct branch; the checkout below confirms it.

```
cd aiw_hg_dagger
git checkout feature-a2
```

A branch named **feature-a2** may already exist in a working copy obtained from a different source. The name does not imply the same history. If a local **feature-a2** is already present, do not assume it corresponds to the branch described here; follow Section 2.3.

2.3 Working copies that already contain a feature-a2

Add this repository as an additional remote rather than overwriting the existing branch. Nothing is lost and the two histories can be compared directly.

```
cd <existing working copy>
git remote add backup https://github.com/da-dawit/aiw_hg_dagger.git
git fetch backup

# What does this branch contain that the local one does not?
git log --oneline feature-a2..backup/feature-a2

# Documentation differences only
git diff --stat feature-a2 backup/feature-a2 -- tmp_readmes/
```

Check out the fetched branch under a distinct name, leaving the existing branch untouched:

```
git checkout -b feature-a2-backup backup/feature-a2
```

3 Installing the training environment

Training is performed off the robot, on a machine with two or more GPUs of at least 40 GB each. The model is loaded in FP32 during training, so the requirement is memory rather than throughput.

3.1 Memory requirement

Component	Memory
Parameters, FP32	12.6 GB
Gradients, FP32	6.5 GB
Optimizer state, FP32	13.0 GB
Total before activations	32.1 GB

3.2 Procedure

```
# 1. Python environment. The project requires Python 3.10 exactly; the
#   dataclass definitions do not parse under 3.11 or later.
curl -LsSf https://astral.sh/uv/uv/install.sh | sh
export PATH="$HOME/.local/bin:$PATH"

# 2. Isaac-GR00T, ROBOTIS fork
git clone https://github.com/ROBOTIS-GIT/Isaac-GR00T-n1.7.git Isaac-GR00T
cd Isaac-GR00T && git checkout e81d02b
uv sync --frozen

# 3. Required patches. Neither is present in a fresh clone.
git apply <hg-dagger>/configs/crop_to_square.patch
git apply <hg-dagger>/configs/mixture_ratio.patch

# 4. Modality configuration
cp <hg-dagger>/configs/ffw_sg2_rev1_16d_config.py \
  examples/CYCLO/ffw_sg2_rev1/

# 5. Hugging Face authentication. The backbone is a gated repository, and
#   the token must be present in the training environment, not only in the
#   login shell: torchrun spawns child processes that do not inherit it.
export HF_TOKEN=<token>
uv run huggingface-cli login --token $HF_TOKEN
```

The two patches are required and are verified by the launcher before training begins. Without `crop_to_square.patch` every camera is letterboxed, which differs from the configuration the base checkpoint was trained under. Without `mixture_ratio.patch` the `path@ratio` syntax is not parsed, and all datasets are collapsed into one specification.

3.3 The base policy

This manual covers the HG-Dagger iteration. It begins from a policy that already performs the task well enough to be worth correcting and improves it from operator corrections. Producing

that policy is separate work (behavior cloning on teleoperated demonstrations) and is not described here.

The reference base policy is checkpoint 30,000 of a sixteen-dimension run over 31 screwing demonstrations. Download it, or an equivalent of your own, before starting.

```
huggingface-cli download dawity/groot_screwing35 \
  --include "16d_30k/checkpoint-30000/*" --local-dir runs/
```

The base policy must have been trained under the image preparation of Section 6.2 and the same sixteen declared dimensions. The continuation inherits both. A base trained under different framing, or over a different modality slice, cannot be warm-started from: the inputs it was fitted to are not the inputs it would now receive.

4 Architecture

The two diagrams on the following pages present the runtime control path and the data path, respectively. Edge labels refer to the identifiers defined in Section 11.

Line	Meaning
Red solid	Human commands the arms
Blue dashed	Policy commands the arms
Black solid	Status, feedback and control plane

Runtime Control Path

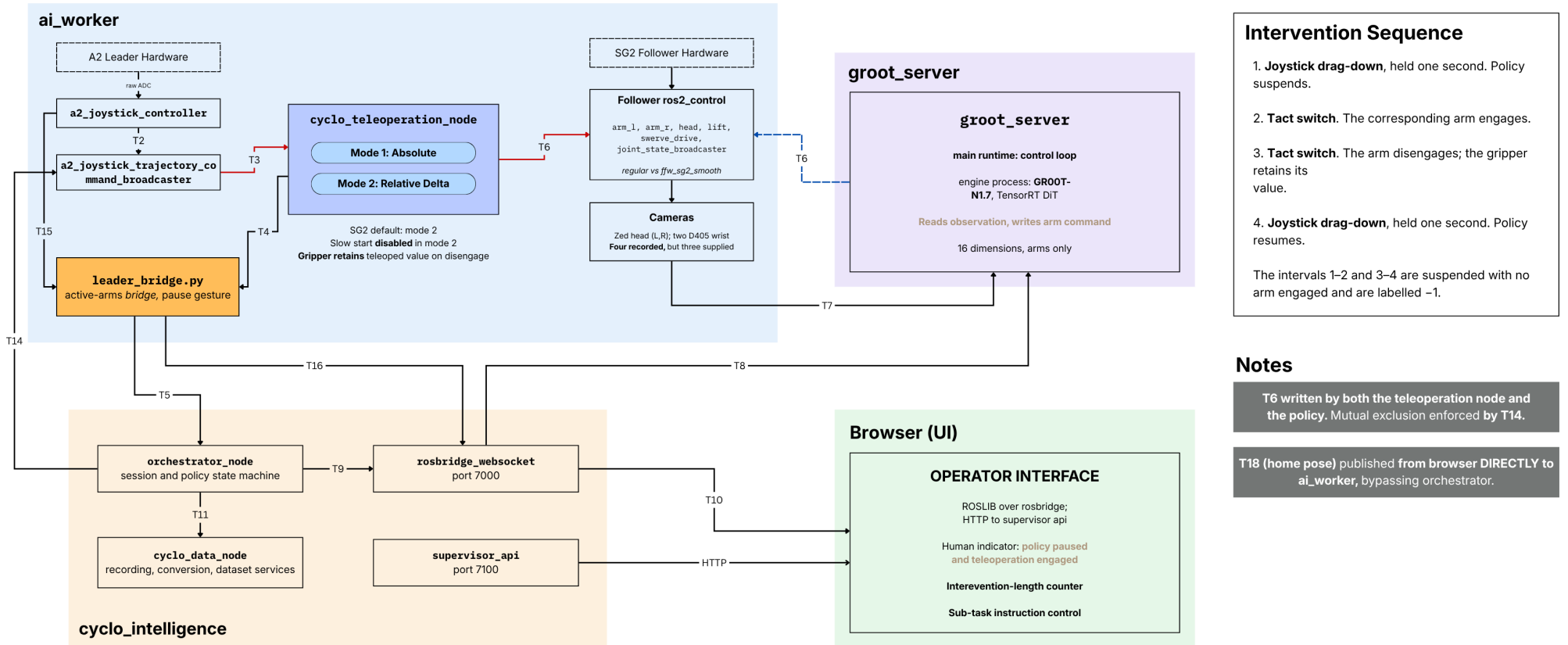


Figure 2: Runtime control path. Which agent is driving the arms at each instant, and the flags that arbitrate between them. Edge labels are the identifiers listed in Section 11.

Recording and Conversion

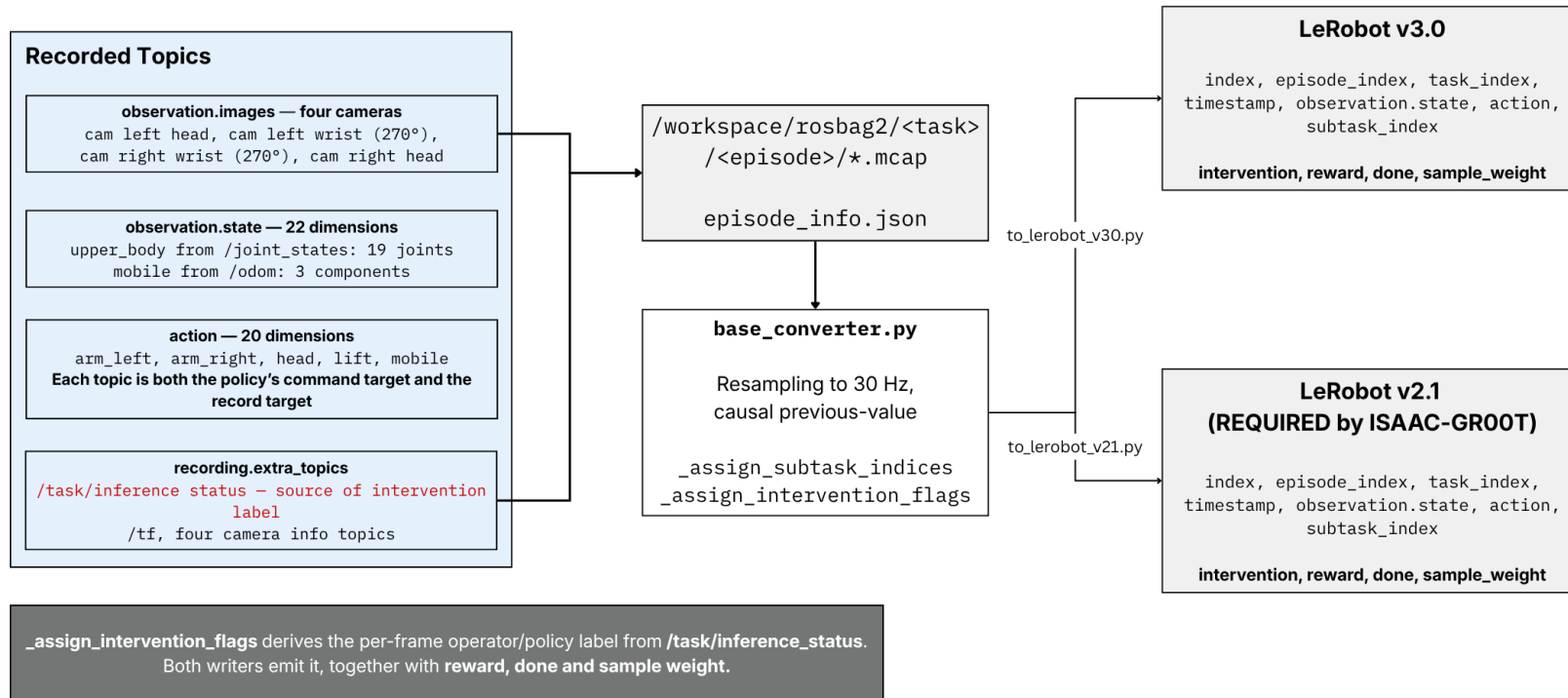


Figure 3: Data path. Recording, conversion, and the point at which the intervention label is attached to each frame.

5 Conducting a session

5.1 Bring-up

The follower is started first, then the leader, then the bridge. The bridge subscribes to topics created by the preceding two.

```
ros2 launch ffw_bringup ffw_sg2_follower_ai.launch.py
ros2 launch ffw_bringup ffw_a2_leader_ai.launch.py
python3 leader_bridge.py
```

The leader and follower **may both remain active for the duration of a session**. They do not contend, because T14 suppresses the leader chain while the policy is driving. Contention between the two indicates that T14 is not being received.

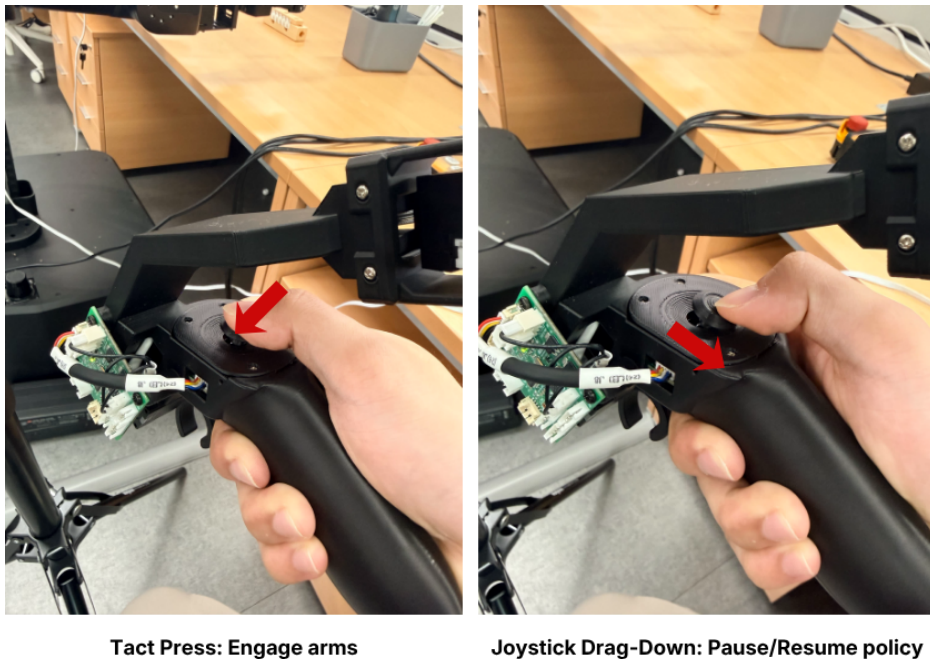


Figure 4: A2 leader, tact switch on each handle, and the joystick whose downward drag suspends and resumes the policy. These are the only two controls used during a session.

5.2 Intervention procedure

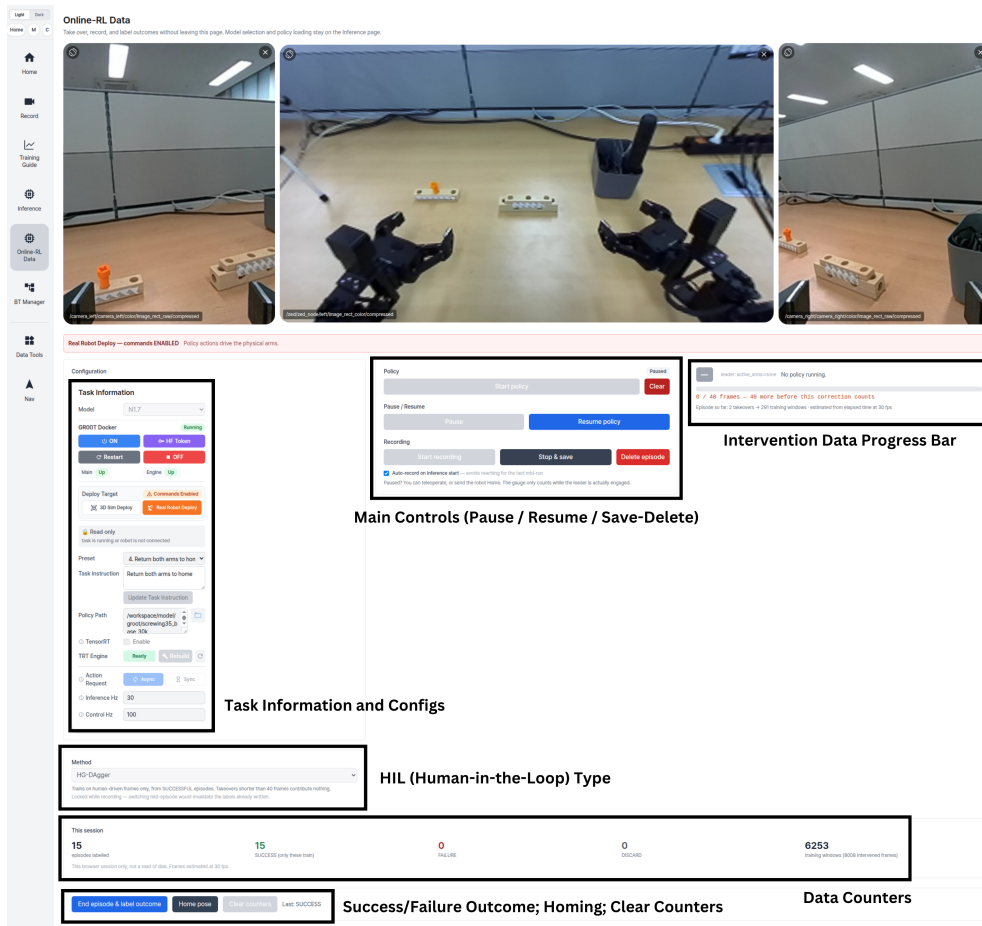


Figure 5: Operator UI Page in Cyclo Intelligence.

The following assumes the policy is executing.

1. Drag the joystick downward and hold for one second. The policy suspends.
2. Press the tact switch for the arm to be corrected. That arm engages.
3. Press the tact switch again. The arm disengages and the gripper retains its teleoperated value.
4. Drag the joystick downward and hold for one second. The policy resumes.

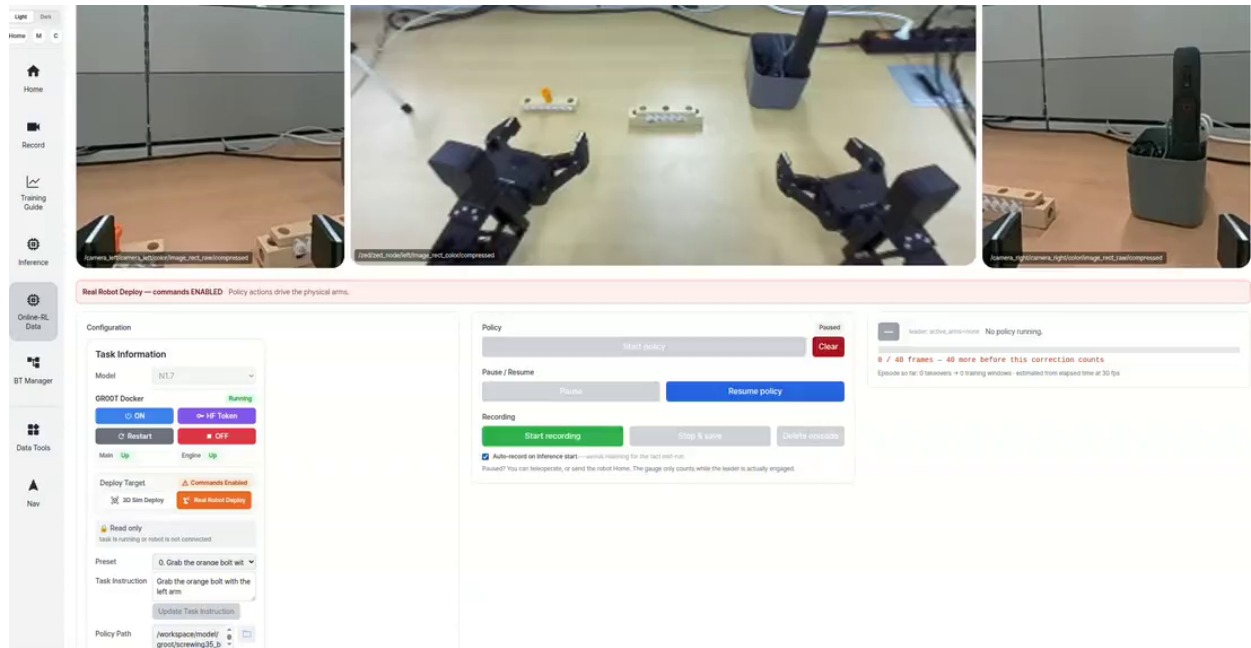


Figure 6: Before the policy is started. The status panel reads NO POLICY RUNNING, the phase chip reads PAUSED, and *Start policy* is the available control. The deploy banner already reads COMMANDS ENABLED, which is a property of the connection and not of the policy: nothing is driving the arms yet.

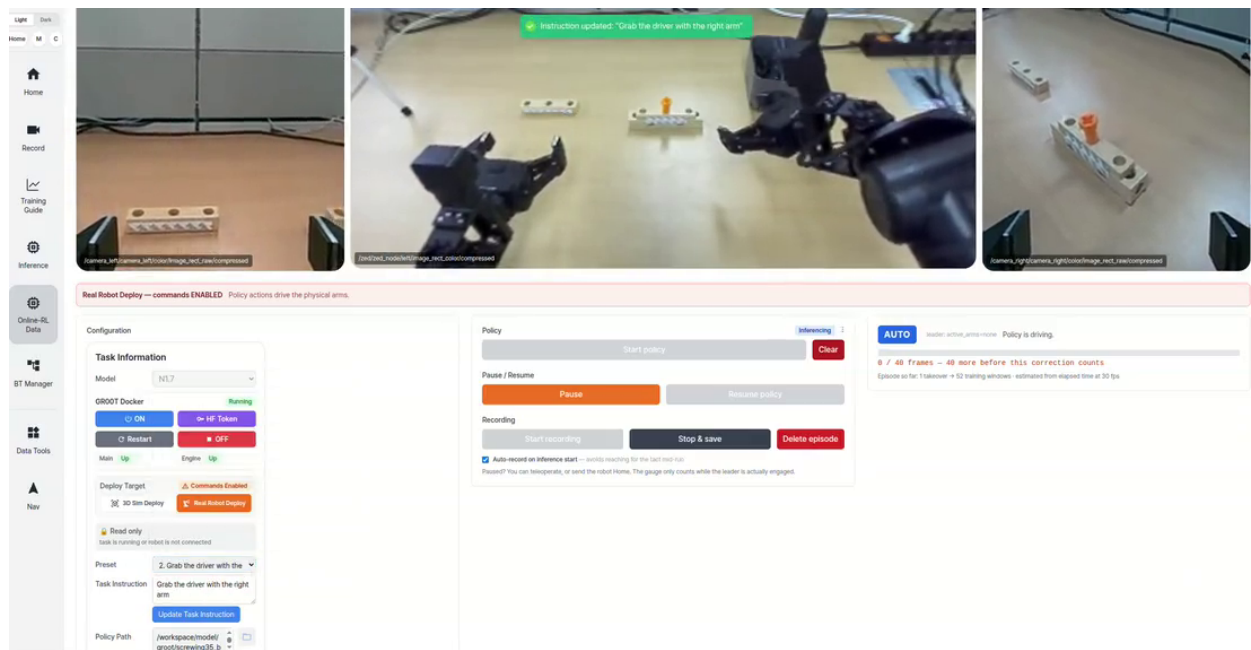


Figure 7: The policy driving. The banner reads AUTO, the phase chip reads inferencing, and *Pause* is the available control. Frames recorded in this state carry label 0. The toast records a sub-task instruction being changed at a phase boundary.

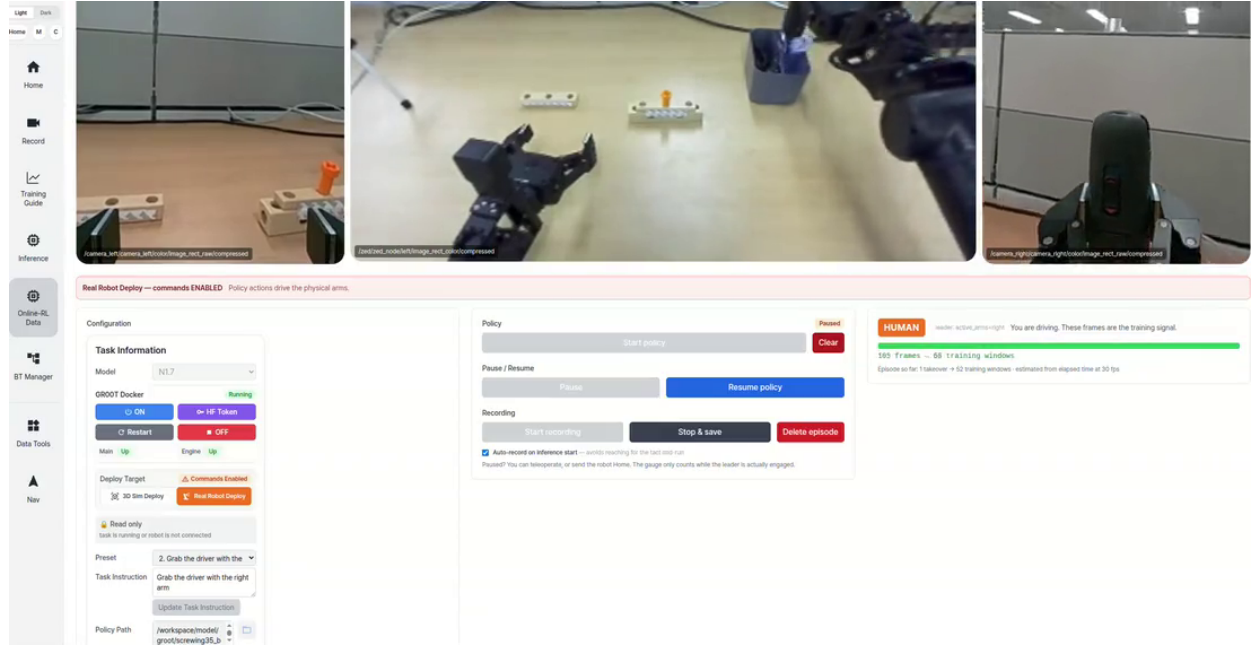


Figure 8: An intervention in progress. The banner reads HUMAN, *Resume policy* is the available control, and the counter reads 105 frames against 66 training windows — one window for each frame from the fortieth onward. Frames recorded in this state carry label 1.

The tact switch is a toggle: engagement and release use the same control. The joystick gesture publishes a single empty message; the orchestrator determines whether it denotes suspension or resumption from the current phase.

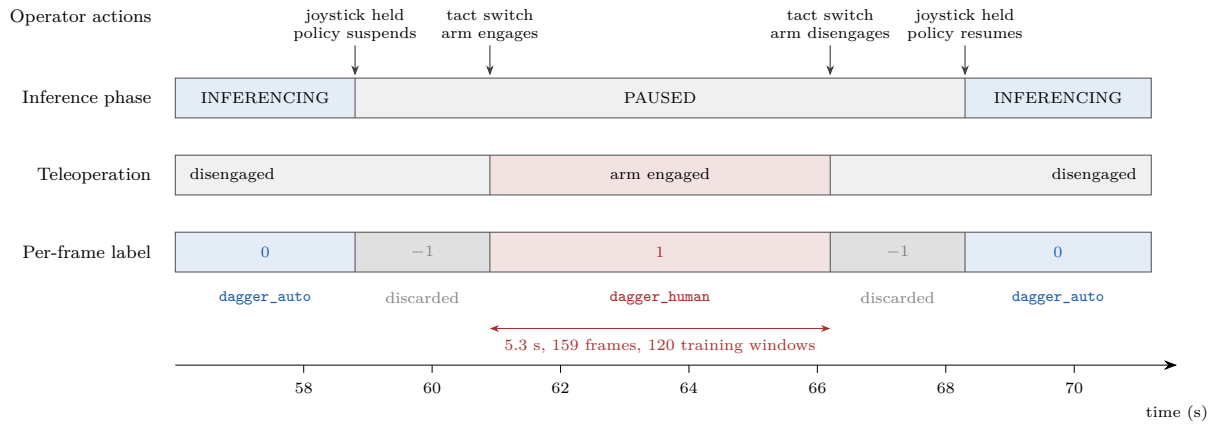


Figure 9: One intervention on a time axis: inference phase, teleoperation engagement, and the resulting per-frame label, showing the excluded intervals either side of the correction.

The intervals between steps 1 and 2, and between steps 3 and 4, have the policy suspended and no arm engaged. These frames are labeled -1 and excluded from training. No urgency is required at these points.

In control mode 2, engagement captures a leader anchor and a follower anchor and thereafter commands `goal = follower_anchor + (leader_now - leader_anchor)`. As the leader moves, the follower moves by the same displacement from where it stopped, so engagement causes no jump. Mode 1 relays absolute leader positions and does jump on engagement, which is why slow start is enabled in mode 1 and disabled in mode 2. SG2 uses mode 2 by default.

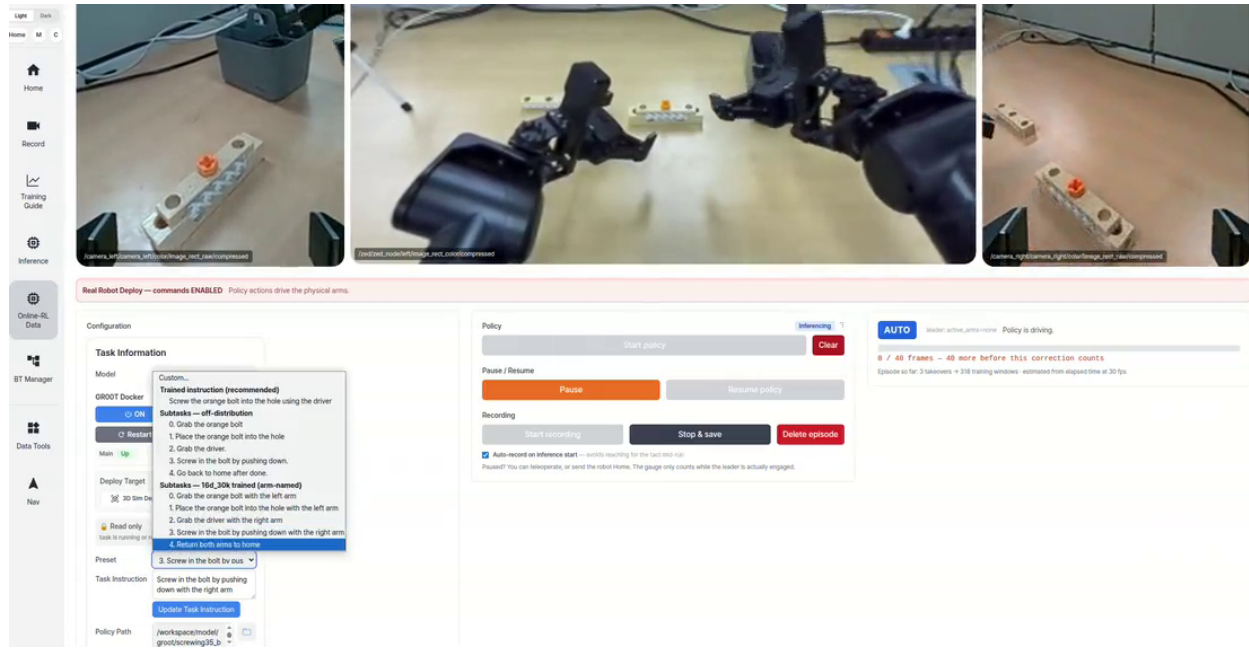


Figure 10: The sub-task instruction control with the list open. The entries correspond to the phases of the task; the selection made at each phase change is what divides an episode into correctly timed sub-segments and is written to `subtask_index`.

5.3 Minimum correction length

The export counts a correction in windows of forty consecutive frames rather than in individual frames, and discards any run shorter than forty frames (`export_dagger_split.py`, `--min-run 40`). A run of n intervened frames therefore yields $n - 39$ windows, and a run shorter than forty frames yields none. The threshold was set for an earlier 40-step model; the current checkpoint's action chunk is sixteen steps, so forty frames is the stricter of the two limits.

Sample yield is strongly nonlinear in correction length. A run of 45 frames yields 6 windows; a run of 120 frames yields 81. A session composed of short corrections may convert without error and produce no training data. At 30 Hz, forty frames is 1.33 seconds; corrections of several seconds are appropriate. The interface displays a counter against this threshold.

5.4 Prohibited actions

- The tact switch must not be used to trigger recording while the policy is executing. Its recording trigger is disabled on this robot for that reason: a single press also engages the

corresponding arm, which would then publish to the topic the policy is driving.

- Episodes must not be saved without an outcome. Aggregation admits successful episodes only; the operator frames of a failed episode are corrections toward an outcome that was not achieved.

6 Recorded data

The robot configuration file is the authoritative specification. Every topic referenced there is subscribed by the recorder, and the observation topics are additionally subscribed by the live inference path. This pipeline is comparable to the rosbag2 recorder Cyclo Intelligence offers.

```
observation:
  images:
    cam_left_head: {topic: /zed/zed_node/left/image_rect_color/compressed}
    cam_left_wrist: {topic: /camera_left/.../compressed, rotation_deg: 270}
    cam_right_wrist: {topic: /camera_right/.../compressed, rotation_deg: 270}
    cam_right_head: {topic: /zed/zed_node/right/image_rect_color/compressed}
  state:
    upper_body: {topic: /joint_states}      # 19 joints
    mobile:     {topic: /odom}              # linear_x, linear_y, angular_z

action:
  arm_left: {topic: /leader/joint_trajectory_command_broadcaster_left/joint_trajectory}
  arm_right: {topic: /leader/joint_trajectory_command_broadcaster_right/joint_trajectory}
  head:     {topic: /leader/joystick_controller_left/joint_trajectory}
  lift:     {topic: /leader/joystick_controller_right/joint_trajectory}
  mobile:   {topic: /cmd_vel}

recording:
  extra_topics:
    - /task/inference_status
    - /tf
    - camera_info topics
```

Listing 1: shared/robot_configs/ffw_sg2_rev1_config.yaml, abridged

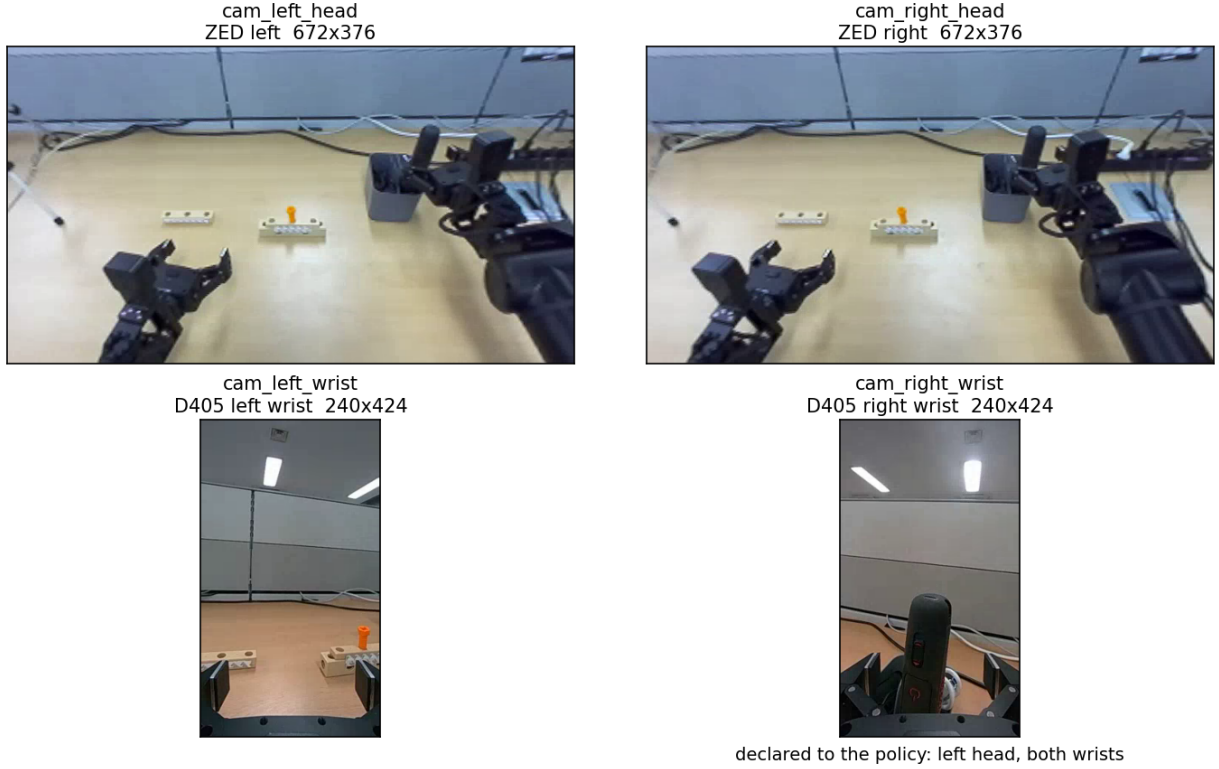


Figure 11: The four recorded camera streams as captured: head left and right, and the two wrist cameras. Three of the four are declared to the policy.

6.1 Dimensions

Field	Dimensions	Composition
<code>observation.state</code>	22	19 upper-body joints, 3 odometry
<code>action</code>	20	16 arm, 1 lift, 3 mobile
Declared to the policy	16	Both arms

The state vector includes `head_joint1` and `head_joint2`; the action vector does not. Slice order follows the insertion order of the configuration file. The policy is trained on the first sixteen dimensions only; the remainder are recorded but unused.

6.2 Image preparation

The vision-language backbone stacks the camera views into a single tensor, so every view presented to it must be square. The four recorded streams are not: the head camera is 672×376 and the wrist cameras are 240×424 . Two methods reconcile this, and the choice between them is not neutral.

Method	Effect
Letterbox	Pad the shorter axis to square. Field of view is preserved; the padding occupies part of the token budget.
Square crop	Crop the longer axis to square. No padding; the cropped region is discarded.

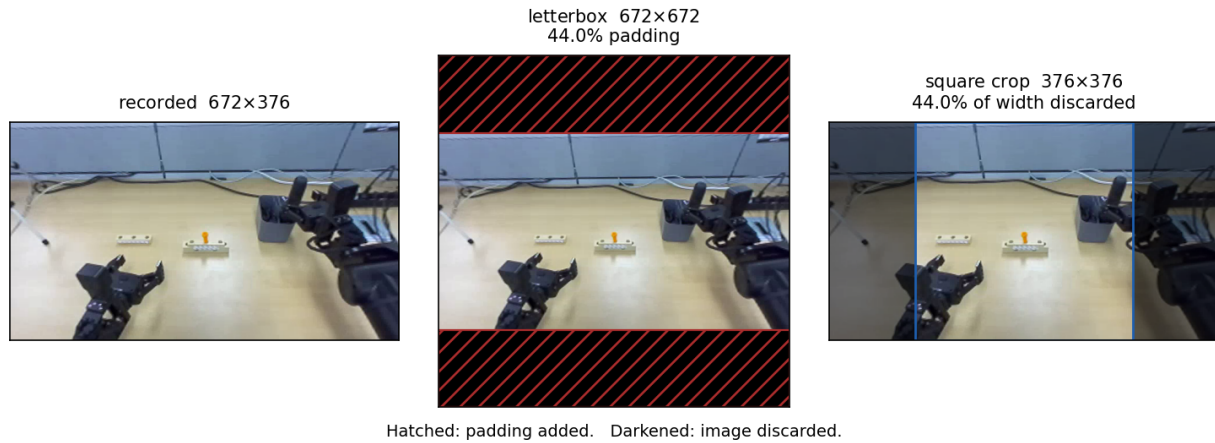


Figure 12: The two methods on one head-camera frame. Hatched: padding added. Darkened: image discarded.

6.3 Why neither method is applied uniformly

Letterboxing every view, which is the default behavior, places an average of 43.6 percent of each model input in black bars. Cropping every view to square removes that waste but discards part of the image. The two camera types were measured separately before the decision was made, and they do not behave alike.

Camera	Measurement
Head	Fixed mounting, so frame-to-frame change is task activity. 47.8 percent of its motion energy , and 67.4 percent during the driver grasp, lies in the lateral bands a centered square crop would remove.
Wrist	Moving mounting, so most frame-to-frame change is background sweeping past the lens; motion energy does not measure task content here. By spatial detail, the band a bottom-anchored crop removes is 43.4 percent of the frame carrying 38.7 percent of its gradient energy , which is close to proportionate. These are also the cameras with the finest resolution on the workspace, at 0.73 to 1.08 mm per pixel.



Figure 13: Shaded regions are what a square crop would remove. Left, the head camera and its column-wise motion energy; right, the wrist camera and its bottom-anchored crop line.

A centered square crop of the head camera would therefore remove the columns in which the work occurs. The wrist cameras carry no equivalent argument in either direction; the case for cropping them is the padding it avoids and the held-out measurement in Section 6.5.

These quantities were measured over episode 0 of the reference session, 3,423 frames, sampled every fourth frame. They are measurements of one episode in one scene and should be regenerated if the cell is rearranged. The regeneration script is `scripts/crop_stats.py`.

6.4 The applied configuration

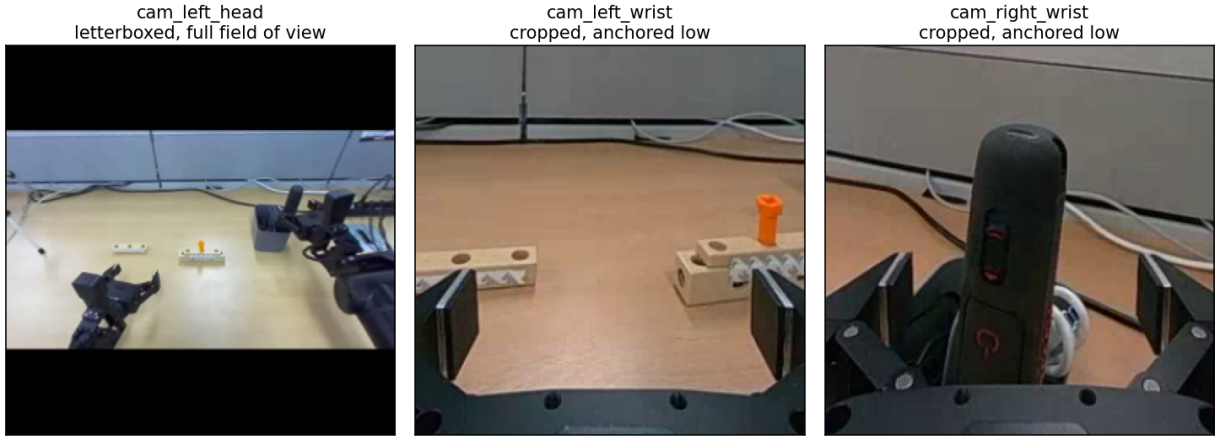
Each view must be square, but the squares need not be of equal size: a trailing resize normalizes all of them to 256×256 before they are stacked.

Camera	Source	Prepared	Padding
<code>cam_left_head</code>	672×376 , full frame	672×672	44.0 %
<code>cam_left_wrist</code>	240×424 , rows 184–424	240×240	0 %
<code>cam_right_wrist</code>	240×424 , rows 184–424	240×240	0 %

The head is letterboxed and retains its full field of view; the wrist crops are anchored to the lower edge, which removes the ceiling. Across all views:

	Uniform letterbox	Applied configuration
Head padding	44.0 %	44.0 %
Wrist padding, each	43.4 %	0 %
Mean over the three views	43.6 %	14.7 %

The saving comes entirely from the wrist cameras; the head view is unchanged. These are geometric quantities and follow from the source resolutions alone.



The three inputs as the model receives them, each 256×256 . Mean black share 14.7%, workspace content 40.3%.

Figure 14: The three views as the model receives them, after preparation and the trailing resize to 256×256 .

6.5 Effect on held-out accuracy

Measured under an identical protocol, on the same held-out episodes, by forward kinematics over the final sixty frames of each phase.

	Uniform letterbox	Applied configuration
Grasp the bolt, left arm	16.3 mm	14.4 mm
Grasp the driver, right arm	22.7 mm	13.3 mm
Systematic fraction, right	0.87	0.40

The systematic fraction is the ratio of the mean bias to the mean error. A value near unity indicates a fixed offset, which no amount of averaging removes; 0.40 indicates that most of the residual is noise. This configuration was in effect for the checkpoint that first grasped reliably on hardware.

The configuration is specified in `configs/roi_hybrid.json` and applied by `crop_to_square.patch`, which is not present in a fresh clone of Isaac-GR00T. Without the patch every camera is letterboxed and the inputs no longer match those the checkpoint was trained on. The training launcher verifies the patch before starting.

6.6 Attention over the prepared views

The action expert cross-attends to the image tokens at eight of the diffusion transformer’s blocks, over four denoising steps. Each view is 8×8 tokens, 192 in all; the values are averaged over heads and over the action queries and aggregated across those 32 captures. Each panel is normalized over its own 64 tokens, so color is comparable within a panel and not between panels.

The same checkpoint was run twice on one frame of the place-in-hole phase, once under the applied crop and once with every view letterboxed. Only the image preparation differs between the two: the frame, the state and the language instruction are identical, and the figure asserts that before it is drawn.

	Applied crop	Uniform letterbox
Normalized entropy over the 192 tokens	0.9851	0.9885
Share held by the 8 strongest tokens	9.1 %	8.4 %
Padding’s share of the token grid	16.7 %	50.0 %
Attention landing on padding	18.9 %	48.8 %

A uniform distribution would give the eight strongest tokens 4.2 percent, and would put exactly as much attention on padding as padding holds of the grid. Both configurations sit close to that: the last two rows differ by 2.2 percentage points under the applied crop and by 1.2 under the letterbox. Letterboxing does not make the policy attend to black bars in preference to the workspace; it gives it three times as many of them to spread over, and the wrist views lose the workpiece to ceiling and wall in the process.

All five phases of the episode were measured, not only the one shown. The additional attention spent on padding under a uniform letterbox ranges from 25.8 percentage points at the bolt grasp to 30.8 at the home pose, against 29.8 here. The effect is a property of the token grid rather than of the frame, so the figure is representative of the episode and not the best case in it.

The checkpoint was trained under the applied crop, so the letterbox rows are the model running on inputs it was never fitted to. They show what the preparation costs, not what a policy trained on letterboxed views would do. Attention this flat does not identify what the policy uses either way, and was not used to choose the crop; the grounds are the motion-energy measurements of Section 6.2 and the held-out error of Section 6.5.

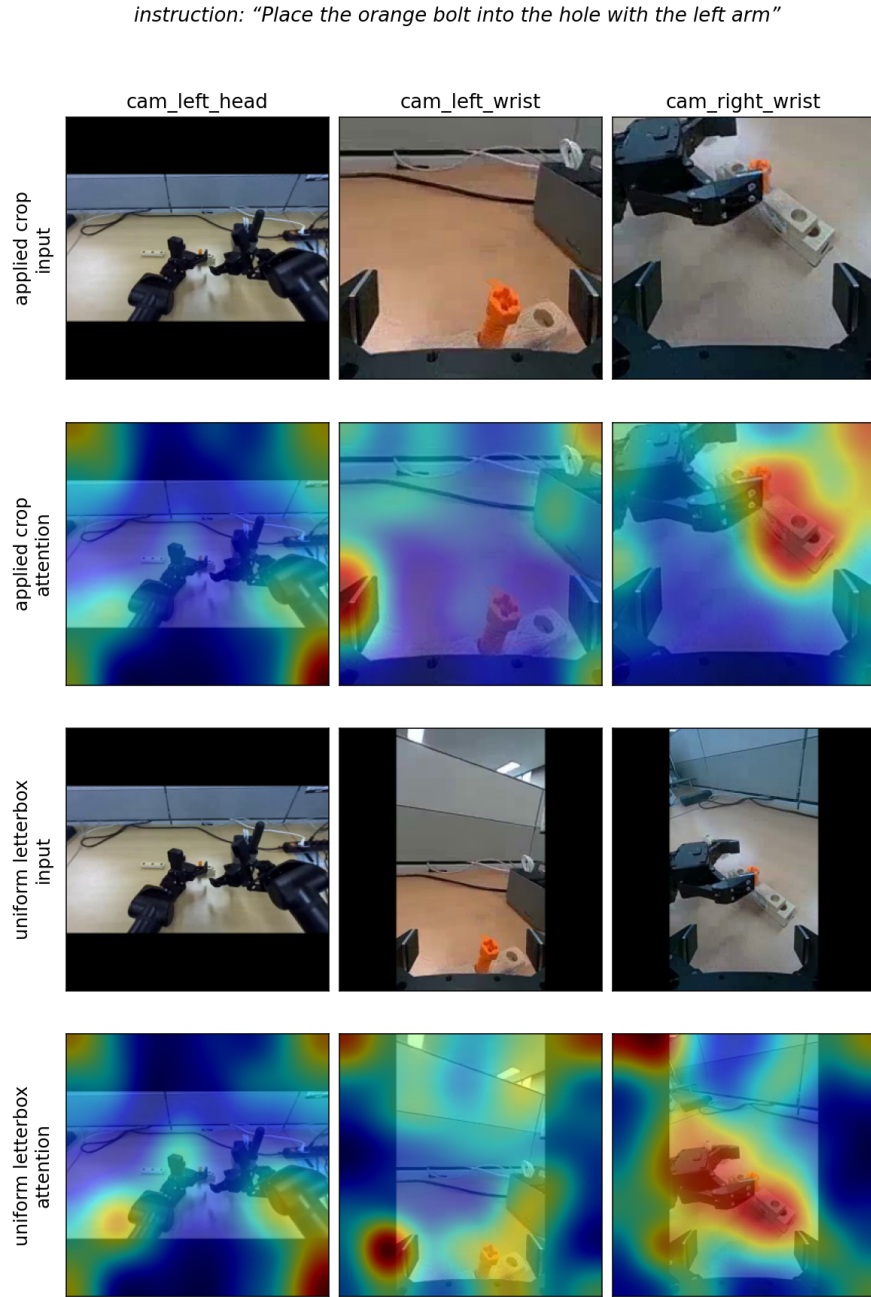


Figure 15: Image cross-attention as the bolt is placed into the hole, checkpoint 30,000, under the two image preparations. Rows one and two are the applied crop, rows three and four the uniform letterbox; within each pair the first row is the views as the model receives them and the second the same views with the attention overlaid. The language instruction, shown above, was the same in both runs. Each panel is min-max normalized over its own 64 tokens, so red marks that panel’s strongest tokens and is not comparable between panels.

7 The intervention label

7.1 Derivation

`/task/inference_status` publishes the inference phase on transitions only. The converter forward-fills it onto the frame grid and assigns:

Condition	Label
Inferencing	0
Paused, teleoperation engaged, not a handoff	1
Paused, teleoperation not engaged	-1
Handoff transient	-1
Prior to the first status message	-1

Label 0 denotes the policy, label 1 the operator, and -1 exclusion.

7.2 Emitted columns

Conversion has two writers, selected by the format flags on T13: `to_lerobot_v21.py` produces LeRobot v2.1, which is the layout Isaac-GR00T requires, and `to_lerobot_v30.py` produces v3.0. Both emit the label, together with three further columns belonging to the online-RL schema. The values are attached to the episode structure during conversion, so neither writer performs any additional computation.

Neither set of columns belongs to stock LeRobot. The format permits additional columns, and this converter already relies on that: `subtask_index`, which divides an episode into sub-segments, is likewise not part of the published specification.

Column	Type	Contents
<code>intervention</code>	int64	1 operator, 0 policy, -1 excluded
<code>reward</code>	int64	Per-frame return
<code>done</code>	bool	Terminal frame of an episode
<code>sample_weight</code>	float32	Per-frame training weight

```
has_hil_feature = any(ep.intervention_flags for ep in episodes)
if has_hil_feature:
    schema_fields.append(pa.field("intervention", pa.int64()))
    schema_fields.append(pa.field("reward", pa.int64()))
    schema_fields.append(pa.field("done", pa.bool_()))
    schema_fields.append(pa.field("sample_weight", pa.float32()))
```

Listing 2: Schema block, common to both writers

The columns are written only where `/task/inference_status` was present in the recording. An ordinary teleoperation session produces none of them, and their absence is not by itself a fault.

Each column is declared in `meta/info.json` as well as written to the parquet files. The declaration is not optional: tooling that reads the feature list will not observe a column present only in the parquet.

Of the four, only `intervention` governs the procedure in this manual. `reward` and `done` belong to reinforcement learning rather than HG-Dagger, and `sample_weight` cannot be acted upon by Isaac-GR00T, which provides no per-frame loss weighting; that weighting is applied at the dataset level instead, as described in Section 8.2. The four are emitted together because they derive from one source.

8 HG-Dagger training

What follows is the continuation run only. It consumes the session recorded in Section 5.2, the original demonstrations the base policy was fitted to, and the base checkpoint of Section 3.3.

8.1 Procedure

The sequence below takes a converted session through to a selected checkpoint. Paths are written relative to the training root.

```
# 1. Validate the converted dataset. Refuses on a degenerate normalizer.
python3 scripts/verify_dataset.py --root datasets/<session> --isaac

# 2. If the export lacks the intervention column, repair it first.
#    Idempotent; backs up whatever it replaces.
python3 scripts/fix_dagger_dataset.py \
    --root datasets/<session> \
    --reference datasets/screwing35_follower_val

# 3. Inspect the partition before writing it.
python3 configs/export_dagger_split.py \
    --src datasets/<session> --out-dir datasets/ --dry-run

# 4. Write the partition. Each intervention becomes its own episode and the
#    video is cut by frame index.
python3 configs/export_dagger_split.py \
    --src datasets/<session> --out-dir datasets/

# 5. Validate both outputs. Video was re-encoded, so the alignment check
#    is the one that matters here.
python3 scripts/verify_dataset.py --root datasets/dagger_human --isaac
python3 scripts/verify_dataset.py --root datasets/dagger_auto --isaac

# 6. Train.
ROOT=$PWD \
BASE=$PWD/runs/16d_30k/checkpoint-30000 \
ORIG=$PWD/datasets/screwing35_follower_train \
HUMAN=$PWD/datasets/dagger_human \
AUTO=$PWD/datasets/dagger_auto \
GR=$PWD/Isaac-GR00T \
NUM_GPUS=2 STEPS=6000 \
bash configs/06_train_dagger.sh

# 7. Select a checkpoint by held-out error, not by training loss.
python3 scripts/checkpoint_sweep.py \
    --runs runs/groot_dagger_it1 \
    --dataset datasets/screwing35_follower_val \
    --v30 datasets/screwing35_follower_val
```

Step 6 prints a mixture line at start-up. It must list three specifications at the intended ratios. A single specification indicates that `mixture_ratio.patch` was not applied and that the datasets are being weighted by size.

8.2 Dataset partition

Isaac-GR00T provides no per-frame loss weighting. The frame-level weighting scheme is therefore approximated at the dataset level using mixture ratios, and the partition is performed at export time. For the reference session:

Dataset	Episodes	Frames	Ratio
Original demonstrations	31	55,701	0.60
Operator corrections	39	6,765	0.30
Policy frames	52	31,804	0.10

Each intervention is exported as an independent episode. Video is indexed per episode and the policy consumes action chunks; concatenating two interventions into one episode would introduce a transition that did not occur.

The policy count is 388 frames below the session’s 32,192 policy-labeled frames (Section 10.2) because the export also drops the two seconds before each takeover and any policy run shorter than forty frames.

In the absence of explicit ratios the loader weights datasets by size. The corrections are 6,765 frames of the 94,270 in the mixture and would therefore receive 7 percent of it rather than 30 percent. The mixture line in the launch log should be checked against the intended ratios.

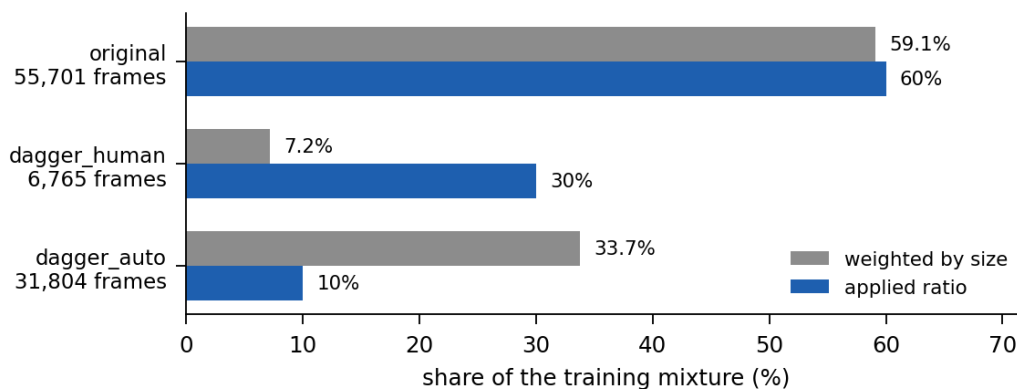


Figure 16: Each dataset’s share of the training mixture: what weighting by size would give, against the ratios actually applied. The original demonstrations barely move, from 59 to 60 percent; the corrections rise from 7 to 30 percent and the policy frames fall from 34 to 10 percent.

8.3 Warm start

The base checkpoint contains weights only. No optimizer state is present and its cosine schedule has annealed to approximately 3×10^{-13} , so there is nothing to resume; the continuation is a fresh

schedule over pretrained weights. The learning rate is the one parameter that must change. At the base run’s 1×10^{-4} a converged checkpoint is driven out of its basin within the first few hundred steps.

Parameter	Base run	Continuation
Learning rate	1×10^{-4}	3×10^{-5}
Global batch	128	128
Frozen	Language and vision	Language and vision
Trained	Projector, diffusion head	Projector, diffusion head
State dropout	0.2	0.2

8.4 Step budget

The step count should be selected by dataset exposure rather than elapsed time.

Steps	Original	Corrections	Duration
1,500	$2.1\times$	$8.5\times$	0.4 h
3,000	$4.1\times$	$17\times$	0.8 h
6,000	$8.3\times$	$34\times$	1.5 h

Exposure is counted as steps $\times$ batch $\times$ the dataset’s mixture ratio, divided by its frame count. The architecture tolerates a good deal of repetition, since diffusion training injects noise at every step and state dropout regularizes further; the base run reached 68.9 epochs over the original data without degrading. The correction set does not have that latitude. Thirty-nine runs from a single scene, repeated several hundred times, are memorized rather than learned, which is what bounds the budget above.

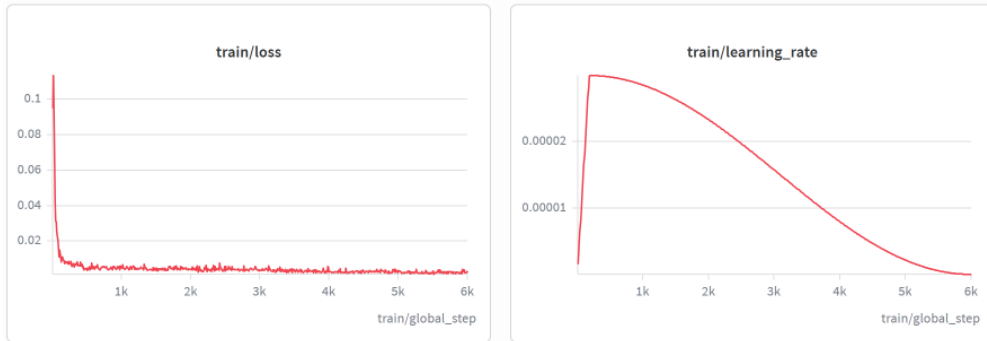


Figure 17: Training loss for the first HG-DAGger iteration ($n = 1$).

8.5 Checkpoint selection

Evaluation must be performed in Cartesian space. A seven-degree-of-freedom arm attains a given position through many joint configurations, so joint-space error is a poor proxy for end-effector accuracy. Select by held-out error rather than by training loss: a small correction set is readily memorised, and training loss continues to fall after held-out performance has turned.

The series below is the base run’s own sweep, shown because it is the one that survives; the procedure is identical for the continuation’s checkpoints.

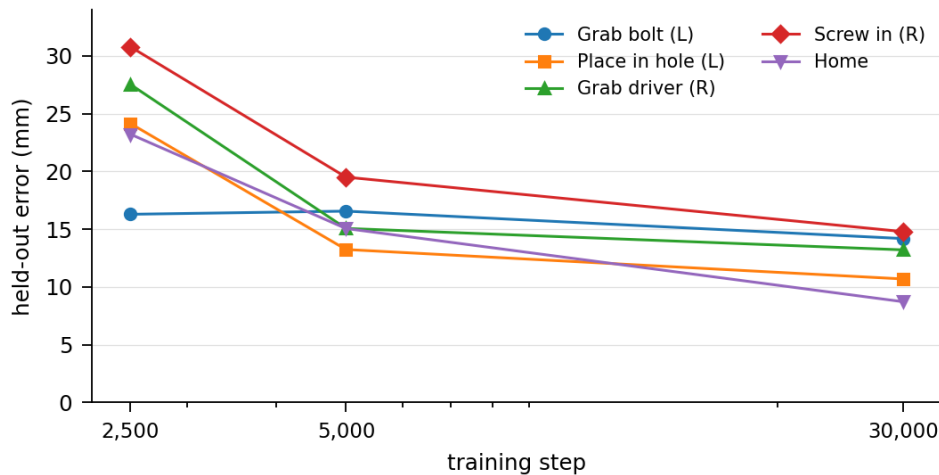


Figure 18: Held-out Cartesian error against training step, one series per task phase, measured by forward kinematics over the last sixty frames of each phase on the four held-out episodes.

9 Deployment

9.1 Transferring the checkpoint

Checkpoints are published to Hugging Face rather than copied between machines.

Repository	Contents
dawity/groot_screwing35	Base checkpoints, including the 16-dimension 30k model
dawity/groot_dagger_it1	Checkpoints from the first HG-Dagger continuation

```
huggingface-cli download dawity/groot_dagger_it1 \
  --include "checkpoint-6000/*" --local-dir /workspace/model/groot/
```

A checkpoint comprises fifteen files. A directory containing fewer than fifteen is an incomplete transfer and will fail to load; the file count should be confirmed before deployment. The results in this manual use the final checkpoint, `checkpoint-6000`; recordings and rollout videos are linked in Section C.

9.2 Configuration that must match training

The following are properties of the checkpoint, not of the deployment, and a mismatch produces degraded behavior rather than an error.

Property	Value	Consequence of mismatch
Declared dimensions	16, both arms	Input is scaled incorrectly
Cameras	3 of the 4 recorded	An unseen view is supplied
Crop	Per-camera, <code>roi_hybrid.json</code>	Framing differs from training
<code>crop_fraction</code>	1.0	Random cropping is reintroduced
<code>use_percentiles</code>	false	Normalization differs
Action horizon	16	Chunk length differs

The last three are read from the checkpoint’s own configuration and are already correct in the published checkpoints. The camera selection and crop are deployment-side settings and must be set to match.

9.3 Bringing the policy up

1. Start the follower, the leader and the bridge as in Section 5.1.
2. Set the policy path in the operator interface to the downloaded checkpoint directory.
3. Load the policy. The phase indicator moves from ready through loading to inferencing.
4. Confirm that commands are enabled before expecting motion. The interface distinguishes a loaded policy from one that is driving the arms.

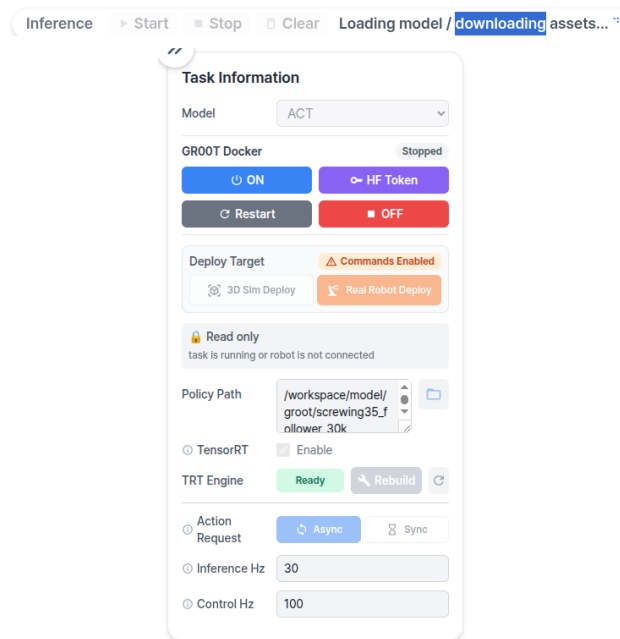


Figure 19: The policy loading. The header reads *Loading model / downloading assets*, the policy path is set to the downloaded checkpoint directory, and the TensorRT engine reports ready. The arms are not yet being driven.

9.4 Control parameters

The deployed runtime, `groot_server`, aligns, resamples and blends each chunk before it reaches the arms. Its settings for this checkpoint are set in `docker/docker-compose.yml` of the Cyclo Intelligence repository.

Parameter	Value	Function
Chunk length	16	Actions returned per inference
Chunk time base	15 Hz	Spacing assumed between actions in a chunk
Control rate	100 Hz	Chunk linearly resampled to one command per tick
Blend	0.08 s	Cross-fade from the last command into a new chunk
Alignment window	0.1 s	Search range for the chunk point closest to the last command

The blend and alignment window are reduced from their defaults of 0.2s and 0.3s, which were

tuned for 40-step chunks. The velocity and acceleration clamps in `spec_sg2.py` (0.6 rad/s and 2.0 rad/s²) belong to the standalone `infer.py` path, which does not drive this robot.

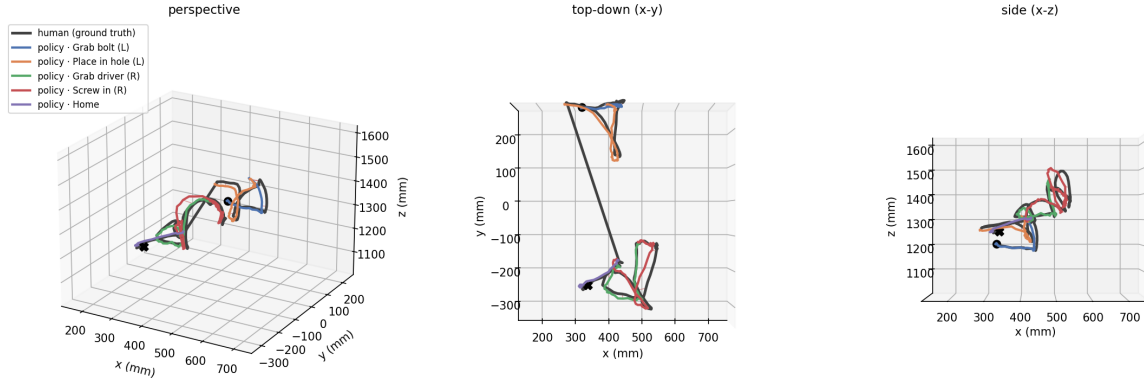


Figure 20: Open-loop Cartesian rollout over one held-out episode, in three views. The dark line is the demonstrated end-effector path and the colored segments are the policy’s, one color per task phase; the thin red segments join each commanded point to the demonstrated one at the same instant.

9.5 Evaluation on hardware

Offline metrics do not establish robot behavior. Held-out evaluation measures agreement with the original demonstrations, and a correction is by definition a departure from them, so a policy that has learned the corrections may score worse. Hardware execution is the only measurement that distinguishes the two cases.

Where a task fails on hardware, the first measurement to take is whether the arm reaches the position the policy commanded. A trace of commanded against achieved position separates a policy error from a control or mechanical one. Additional training data cannot correct the latter.

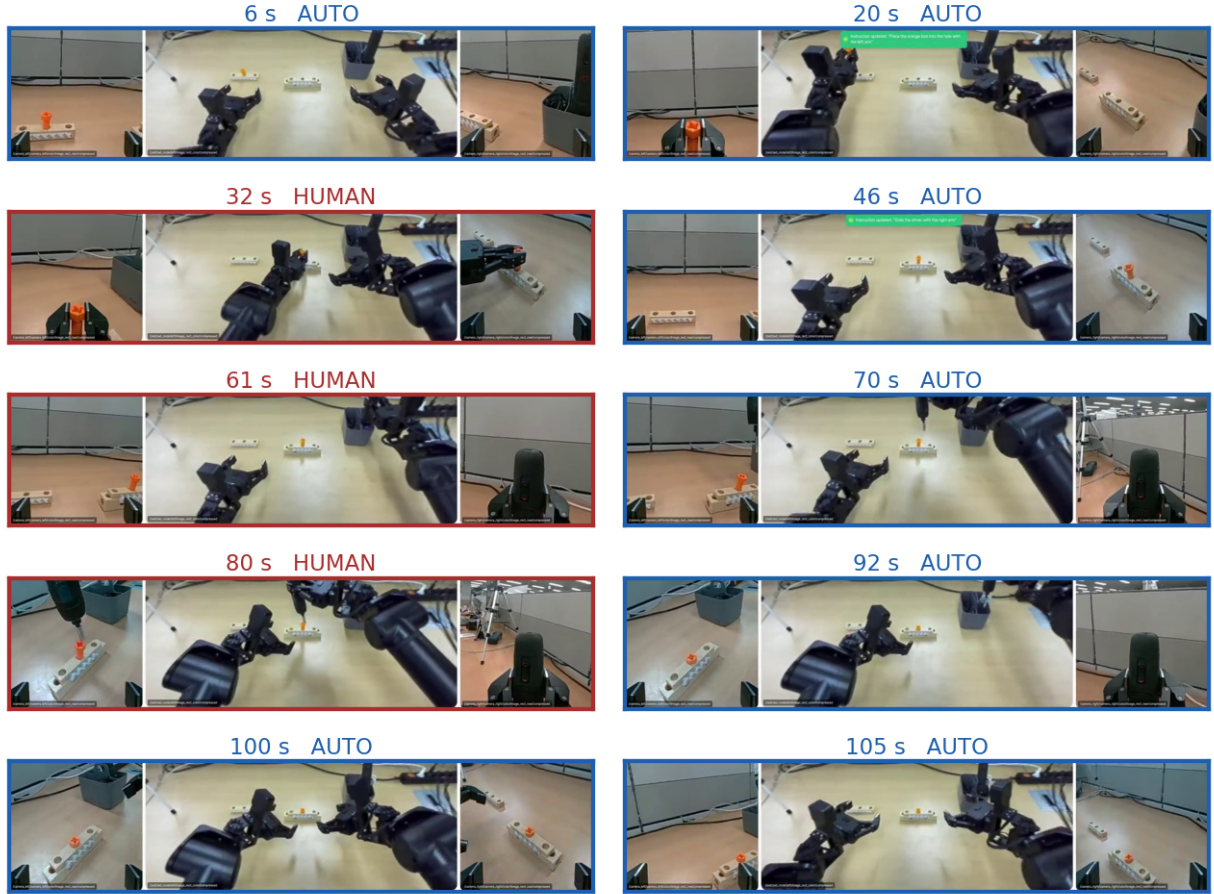


Figure 21: The session on the physical robot, sampled across its 116 seconds. Each panel shows the three camera views at that instant, bordered and labeled by which side was driving. The three red panels fall inside the operator’s takeovers, at 31–34, 58.5–64.5 and 78–84 seconds.

One HG-Dagger round (13 corrected episodes and 6,000 training steps on top of the 30,000-step base policy) made the long-horizon bolt-screwing task succeed end to end. Each value below is the share of all 30 trials that completed that stage.

Stage	Base policy	After one round
Grasp the bolt	100 %	100 %
Place the bolt in the hole	40 %	100 %
Grasp the driver	10 %	87 %
Screw in the bolt	0 %	40 %
Place and return	0 %	40 %

Every trial that finished screwing also completed place and return (100 % of those trials), so end-to-end success rose from 0 % to 40 %. Higher rates will need more correction episodes.



Figure 22: The bolt seated in the block on the physical robot, both arms clear of the workpiece at the end of the episode.

10 Verification

10.1 Dataset validation

```
python3 scripts/verify_dataset.py --root <dataset> --isaac
```

Check	Detects
Action frame offset	Action misaligned from the state it follows
Freeze frames	Stationary arm under a label commanding motion
Label consistency	Inconsistent phase semantics across episodes
Camera shapes	Unannounced resolution change
Modality declaration	Dimensions declared but not supplied
Normalizer	Declared dimension of zero range
Dataset version	Layout not matching the required version
Video alignment	Video and parquet disagreeing after re-encoding

```

verifying /home/robotis/robot_aiworker/datasets/dagger_human
=====
1. ACTION FRAME      mean |action[t] - state[t+5]| =  0.073 deg
   -> consistent with follower[t+5]
2. FREEZE FRAMES    0 of 6765 (0.00%) -- at least one arm frozen in a sustained freeze (>=15f)
   while the label commands motion, PER ARM
3. LABEL CONSISTENCY worst across-episode SD = 0.06 deg (subtask 2)
4. TASKS.JSONL (v2.1) first two = ['Grab the orange bolt with the left arm', 'Place the orange bolt
   into the hole with the left arm']
5. CAMERA SHAPES
   cam_left_head      376x672
   cam_left_wrist     424x240
   cam_right_head     376x672
   cam_right_wrist    424x240
6. MODALITY.JSON    present
   arm_left   [ 0, 8) ok
   arm_right  [ 8,16) ok
   declared dims: 16 of 22
10. NORMALISER      0 declared dim(s) with a degenerate range
8. DATASET VERSION  v2.1
9. VIDEO ALIGNMENT  0 of the per-episode videos disagree with their parquet

   6765 frames, 39 episodes (35 train / 4 held out)
=====
WARN no --val-root given. Isaac's eval_strategy defaults to "no" and val_dataset_path is not
   exposed by launch_finetune's CLI, so without a separate val dataset AND a launcher patch the run
   trains blind.

All checks passed. Safe to train.

```

Figure 23: A passing `verify_dataset` report over `dagger_human`. The normalizer and video-alignment checks are the two that most often fail. The closing warning concerns the absence of a separate validation set, not a fault in the dataset.

10.2 Post-conversion assertion

```

import pandas as pd, glob, json
df = pd.concat([pd.read_parquet(f)
                for f in glob.glob("<ds>/data/**/*.parquet", recursive=True)])
assert "intervention" in df.columns
assert "intervention" in json.load(open("<ds>/meta/info.json"))["features"]
print(df.intervention.value_counts())

```

All three label values should be present. On the reference session they were 6,765 operator frames, 32,192 policy frames and 4,729 excluded, of 43,686.

Two conventions exist and they are inverses. `intervention` uses 1 for the operator and 0 for the policy; `task_is_policy`, written by older exports, uses 1 for the policy and 0 for the operator. Both use `-1` for exclusion. A dataset carrying the second fails the assertion above rather than passing on the complement of the intended frames, which is why the assertion names the field.

11 Topic and service reference

Numbered notes are given in Section 11.1.

ID	Kind	Name	Function	Note
T1	Topic	<code>tact_trigger</code>	Tact switch event	1
T2	Topic	<code>joystick_command</code>	Arm engage toggle	2
T3	Topic	<code>raw_joint_trajectory</code>	Leader arm command	
T4	Topic	<code>control_status</code>	Active arms and mode	3
T5	Topic, latched	<code>active_arms_str</code>	Active arms, string form	3
T6	Topic	<code>joint_trajectory</code>	Arm command to the follower	4
T7	Topic	<code>joint_states</code> , cameras	Observation	5
T8	Topic	<code>engine_command</code>	Policy load and run	
T9	Service	<code>/task/command</code>	Policy and recording control	
T10	Topic, latched	<code>inference_status</code>	Inference phase	6
T11	Service	<code>/data/recording</code>	Recording lifecycle	
T13	Service	<code>/data/convert</code>	Dataset conversion	7
T14	Topic, latched	<code>policy_active</code>	Leader suppression	4
T15	Topic	<code>sensorxel_joy_values</code>	Raw joystick axes	
T16	Topic	<code>toggle_pause_request</code>	Suspend or resume	8
T17	Topic	<code>control_command</code>	Control mode selection	9
T18	Topic, latched	<code>home_in_progress</code>	Home pose	10
T19	Service	<code>/task/command</code>	Sub-task instruction update	11

11.1 Notes

1. The recording trigger is disabled. A single press additionally engages the corresponding arm.
2. Supports enable, disable and toggle. The tact switch issues a toggle.
3. T4 is not recorded, as its message package is unavailable to the recorder. T5 carries equivalent information in a universally loadable type and should be recorded in its place.
4. T6 is written by both the teleoperation node and the policy. T14 suppresses the leader chain while the policy is active.
5. Four cameras are recorded; three are declared to the policy.
6. Phase values: ready 0, loading 1, inferencing 2, paused 3. Published on transitions only; the converter forward-fills. Sole source of the intervention label.
7. Both format flags set false runs both conversions.
8. Triggered by a one-second drag-hold. A single topic with two meanings, resolved by the current phase.
9. Mode 1 is absolute; mode 2 is relative delta. SG2 uses mode 2.

10. Published from the browser directly, bypassing the orchestrator.
11. Appends to the instruction change log, which divides the episode into correctly timed sub-segments.

12 Fault diagnosis

Symptom	Cause and remedy
Leader and policy contend for the arms	T14 is not being received.
All frames carry one sub-task label	The instruction was not updated during the take. Use the instruction control at each phase change.
Interface indicator inconsistent with the robot	The indicator is a conjunction of policy state and teleoperation state; a stale input yields an incorrect indicator.
An intervention produced no training samples	The run was shorter than forty frames.
Dataset lacks the intervention column	Either the recording carried no <code>/task/inference.status</code> , in which case there was no policy to intervene in, or the export predates the writer emitting it. See Section 7.
Training loss decreases as behavior degrades	Expected with a small correction set. Select by held-out Cartesian error.
Video files empty after conversion	Conversion preceded completion of the recording flush. Confirm the bag contains frames and re-run.

A Glossary

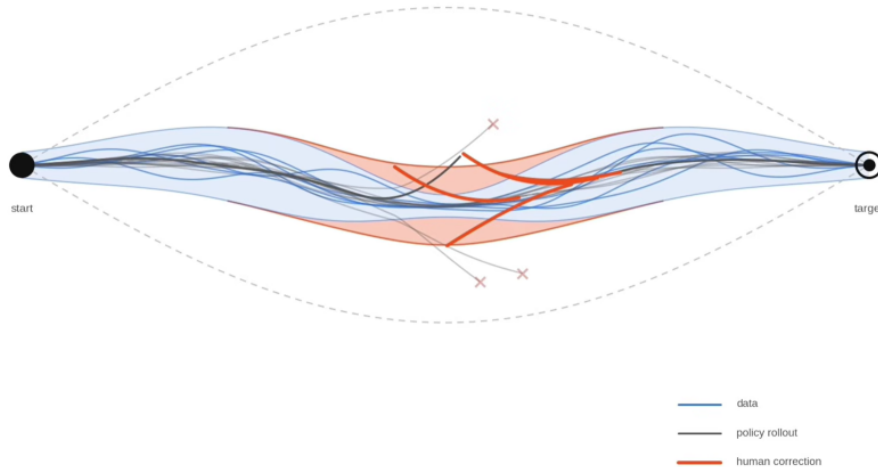


Figure 24: The HG-Dagger loop.

HG-Dagger

Human-gated DAgger. The policy executes and the operator determines when to intervene. Only intervened frames are treated as expert data, and they are aggregated with the original demonstrations rather than replacing them.

Action chunk

The block of future actions produced by one inference. Sixteen steps in this system. The export's forty-frame minimum on corrections is a separate, stricter threshold.

Latched topic

A topic on which a subscriber receives the current value upon connection.

Relative-delta teleoperation

Control mode 2. The follower is commanded by the leader's displacement since engagement rather than to its absolute pose.

Open-loop evaluation

Evaluation in which recorded observations are replayed through the policy without its output affecting the robot.

B Reference quantities

Measured values from the reference session and base training run. These are measurements rather than constants and should be regenerated rather than quoted.

Recording rate	30 Hz
Action chunk	16 steps
Minimum exported correction	40 frames, 1.33 s
Base policy warm-started from	30,000 steps
Original demonstrations	31 episodes, 55,701 frames
Session recording	13 episodes, 43,686 frames
Recovered interventions	39
Operator proportion of the session	15.5 %
Held-out validation set	4 episodes, 6,962 frames

C Links

Repository	Contents
<code>aiw_hg_dagger</code>	Robot-side packages, branch <code>feature-a2</code> ; this manual and its figures under <code>tmp_readmes/</code>
<code>cyclo_intelligence_hg_dagger</code>	Orchestration, recording, dataset conversion, operator interface
<code>hg-dagger</code>	Training, evaluation and dataset tooling; the two Isaac-GR00T patches

Session recordings, checkpoints and rollout videos: [Google Drive](#).